

Information Retrieval

Mid-Semester Test · Complete Study Notes

CLOSED BOOK

2 HOURS

30% WEIGHT

SESSIONS 1-8

WHAT'S INSIDE

01

Introduction to IR

IR vs data · models · process

02

Boolean Retrieval

incidence matrix · inverted index

03

Preprocessing & Postings

tokens · stemming · phrases

04

Dictionaries & Tolerant

wildcards · edit dist · Soundex

05

Index Construction

BSBI · SPIMI · MapReduce

06

Index Compression

Heaps · Zipf · VB codes

07

Vector Space Model

tf-idf · cosine · ranking

★

Cheat Sheet

formulas · traps · checklist

Compiled from the CS1-CS8 lecture slides and Manning, Raghavan & Schütze, Introduction to Information Retrieval (T1). Worked numericals mirror the in-slide exercises.



High-Yield Map

Where to spend your revision time

The mid-sem is **closed book**, so the algorithms you must be able to **run by hand** matter most. The slides repeat certain exercises — those are the strongest signal for what to drill. Below: the numerical/algorithmic skills first, then the concepts most likely to appear as short-answer or compare-and-contrast questions.

Must be able to compute by hand

Priority	Skill	Where it lives
★★★★	Inverted index + Boolean merge	Ch 2 — incidence matrix → postings → AND/OR/NOT
★★★★	Levenshtein edit distance (DP matrix)	Ch 4 — fill the grid, read bottom-right
★★★★	tf-idf + cosine similarity	Ch 7 — the Inc.ltn table, novels cosine
★★★★	Soundex code of a name	Ch 4 — 4-char phonetic hash
★★★★	Variable Byte encode / decode	Ch 6 — gaps → bytes and back
★★	Boolean query optimisation order	Ch 2 — smallest postings / OR-size first
★★	Dictionary blocking space saving	Ch 6 — $(k-1) \cdot 3$ saved, k spent
★★	Skip-pointer comparison count	Ch 3 — $\sqrt{P}$ spacing
★★	Permuterm rotation for a wildcard	Ch 4 — move * to the end after \$
★★	k-gram overlap + Jaccard	Ch 4 — shared k-grams / union

Must be able to explain / compare

Priority	Concept
★★★★	IR vs. Data Retrieval; the three classic models (Boolean / Vector / Probabilistic)
★★★★	Inverted index structure: dictionary + postings; why not the incidence matrix
★★	Tokenisation issues; token vs type vs term; stemming vs lemmatisation; Porter
★★	Dictionary structures: hashtable vs tree vs B-tree (prefix problem)
★★	BSBI vs SPIMI; MapReduce map/reduce schema; dynamic indexing & log merge
★★	Heaps' law and Zipf's law (what each models)
★	Precision vs recall; why Boolean gives no ranking; VSM limitations

● **EXAM TIP**

When a question says “show your working”, the marks are in the trace table, not the final number. Always draw the DP grid, the postings merge, or the tf-idf table in full.

01

MODULE 01

Introduction to IR

CS1 · What IR is and the classic models

1.1 What is Information Retrieval?

Information Retrieval (IR) is finding material — usually **documents** — of an **unstructured** nature (usually text) that satisfies an **information need** from within large collections. It is not restricted to web search: email search, searching a laptop, corporate knowledge bases and legal retrieval are all IR.

IR is about the **representation, storage, organisation of, and access to** information items. The focus is on the user's *information need* rather than a precisely specified query.

Ad-hoc retrieval A one-time query against a **static** collection (e.g. a Google search).

Filtering A static user profile applied to a **dynamic** stream of incoming documents.

Retrospective “Searching the past” — varied queries on a static, time-invariant collection.

Prospective “Searching the future” — a static query on a dynamic, time-dependent collection.

1.2 IR vs. Data Retrieval ★★★

A favourite short-answer question. Data retrieval (e.g. SQL on a database) wants *which records exactly match*; IR wants *information about a topic* and tolerates small errors.

Aspect	Data Retrieval	Information Retrieval
Data	Structured	Unstructured (free text)
Fields	Clear semantics (SSN, age)	No fields other than text
Queries	Defined — SQL, relational algebra	Free text / natural language, Boolean
Matching	Exact — results always “correct”	Imprecise — must measure effectiveness
Errors	One wrong object = failure	Small errors tolerated

1.3 The IR task and relevance

Input: a corpus of natural-language documents + a textual query string. **Output:** a **ranked** set of documents relevant to the query.

Relevance is a *subjective* judgment: on the right subject, timely, authoritative, and satisfying the user's intended use of the information.

1.4 The retrieval process — system components

- **Text operations** form index tokens — stop-word removal, stemming.
- **Indexing** builds an **inverted index** mapping words → document pointers.
- **Searching** retrieves docs containing a query token from the inverted index.
- **Ranking** scores retrieved docs by a relevance metric.
- **Query operations** transform the query — expansion, relevance feedback.
- **User interface** manages query input, result display, feedback and visualisation.

1.5 Classic IR models ★★★

Traditional IR uses **index terms** to retrieve and **rank** documents. The importance of term k_i in document d_j is captured by a weight w_{ij} . Different premises about relevance give different models.

Model	Basis	Weights	Key idea
Boolean	Set theory & Boolean algebra	binary {0,1}	Doc is a set of terms; query is AND/OR/NOT expression
Vector Space	Linear algebra	continuous (tf-idf)	Docs & query are vectors; rank by similarity
Probabilistic (BIR)	Probability theory	binary	Rank by probability of relevance; refine by feedback

Boolean model — strengths & limits

ADVANTAGES

- ▶ Clean formalism, neat semantics
- ▶ Easy to implement
- ▶ Intuitive; precise match
- ▶ Dominant for document DB systems

LIMITATIONS

- ▶ No partial matching (binary decision)
- ▶ No ranking / grading of results
- ▶ Hard to express need as Boolean
- ▶ Returns too few or too many docs

Vector model — strengths & limits

ADVANTAGES

- ▶ Non-binary term weights (tf-idf)
- ▶ Continuous degree of similarity
- ▶ Ranks docs; allows partial matching
- ▶ Efficient for large collections

LIMITATIONS

- ▶ Assumes terms are independent
- ▶ Word order lost (bag of words)
- ▶ Long docs poorly represented
- ▶ Weighting intuitive but informal

● PROBABILISTIC MODEL IN ONE LINE

Binary Independence Retrieval (BIR): guess an initial ideal answer set, let the user mark retrieved docs relevant/non-relevant, then iteratively refine the probability estimate. Docs ranked by decreasing probability of relevance; weights are binary; assumes term independence.

1.6 Logical view of documents

A document is reduced from full text to a set of **index terms / keywords**. Two key reductions:

Stop-words

Function words (of, in, the, with...) with little IR value; removing them via a **stop-list** usually improves effectiveness and saves space.

Stemming

Reduce word forms to a common grammatical root — compute, computes, computing, computation → **comput**. Creates a standard representation for related terms.

02

MODULE 02

Boolean Retrieval

CS2 · Incidence matrix, inverted index, query merge

2.1 The motivating problem

Find all Shakespeare plays containing **BRUTUS AND CAESAR AND NOT CALPURNIA**. A **linear scan** (grep) works for tiny collections but is too slow for billions of words, can't do flexible matching, and gives no ranking. Solution: **index the documents beforehand**.

2.2 Term-document incidence matrix

A matrix with terms as rows and documents as columns; entry $(t, d) = 1$ if term t appears in document d , else 0. Each **term vector** is a row read across the documents.

Brutus AND Caesar AND NOT Calpurnia → bitwise AND of the three rows

It works but the matrix is **extremely sparse** and huge ($|\text{terms}| \times |\text{docs}|$) — wasteful to store all the zeros. **Record only what occurs** → the inverted index.

2.3 The inverted index ★★★

For each term t , store a list of all documents that contain it. The structure splits into:

Dictionary the sorted vocabulary; for each term, its document frequency and a pointer to its postings.

Postings list the list of docIDs containing the term, **sorted by docID**; stored as a linked list or variable-length array.

Building the inverted index — 4 steps

- 1 **Collect** the documents to be indexed.
- 2 **Tokenise** the text — turn each document into a list of tokens.
- 3 **Linguistic preprocessing** — normalise tokens to indexing terms (case-fold, stem, stop-words).
- 4 **Index**: produce (term, docID) pairs, sort by term then docID, merge duplicates, add document frequency, split into dictionary + postings.

2.4 Processing Boolean queries

To answer **Brutus AND Calpurnia**: look up each term in the dictionary, retrieve its postings list, then **intersect** the two lists.

The merge (intersection) algorithm

Walk both postings lists with two pointers. If the docIDs are equal, emit it and advance both; otherwise advance the pointer on the **smaller** docID. Because both lists are sorted, this runs in **$O(m + n)$** for lists of length m and n .

● WHY SORTED POSTINGS MATTER

The linear-time merge only works because postings are kept in increasing docID order. This single property underpins almost all of Boolean IR.

2.5 Query optimisation ★★

- **AND of several terms:** process in order of **increasing document frequency** — start with the smallest postings list, intersect, and keep cutting the candidate set down.
- **OR groups:** estimate each OR's size by the **sum** of its terms' document frequencies, then process the conjunction in increasing order of those OR-sizes.

● WORKED RULE OF THUMB

For **Brutus AND Caesar AND Calpurnia** with Calpurnia rarest, execute as (Calpurnia AND Brutus) AND Caesar — smallest intermediate result first.

2.6 Beyond exact match — Westlaw & proximity

Boolean was the primary commercial retrieval tool for three decades (email, library catalogs, macOS Spotlight). **Westlaw**, the largest legal search service, kept Boolean (“Terms and Connectors”) as default. It adds **proximity operators**:

/n within n words

/s within a sentence

/p within a paragraph

SPACE disjunction (OR), not conjunction

2.7 Evaluating Boolean results — precision & recall

Let A = relevant docs in the collection, C = total docs retrieved, B = relevant docs retrieved.

$$\text{Precision} = B / C \quad \text{Recall} = B / A$$

Precision What fraction of the **returned** results are relevant?

Recall What fraction of the **relevant** documents were returned?

● BOOLEAN WEAKNESS

Binary matching gives **no ranking**, so precision/recall are about the set as a whole — there's no notion of “best” document. This motivates the Vector Space Model.

N1

MODULE N1

Worked Numericals · Boolean

Inverted index, merge, optimisation

Numerical 1 · Build the inverted index

● SETUP

Doc1: breakthrough drug for schizophrenia · Doc2: new schizophrenia drug · Doc3: new approach for treatment of schizophrenia · Doc4: new hopes for schizophrenia patients. Build the term-document incidence matrix and the inverted index.

Term	Incidence (D1 D2 D3 D4)	Postings (sorted docIDs)
breakthrough	1 0 0 0	→ 1
drug	1 1 0 0	→ 1 → 2
schizophrenia	1 1 1 1	→ 1 → 2 → 3 → 4
new	0 1 1 1	→ 2 → 3 → 4
approach	0 0 1 0	→ 3
treatment	0 0 1 0	→ 3
hopes	0 0 0 1	→ 4
patients	0 0 0 1	→ 4

● INSIGHT

The dictionary holds each term once with its document frequency; the postings list holds the docIDs in increasing order. Stop-words (for, of) are dropped. Note how sparse the matrix is — the inverted index stores only the 1s.

Numerical 2 · Boolean merge

● SETUP

france → 1 2 3 4 5 7 8 9 11 12 13 14 15 ; paris → 2 6 10 12 14 ; japan → 12 15. Compute ((paris AND NOT france) OR japan).

#	Operation	Result
1	paris AND NOT france (paris docs not in france)	6, 10
2	(result) OR japan	6, 10, 12, 15
ANSWER = {6, 10, 12, 15}		

● INSIGHT

paris = {2,6,10,12,14}; remove those also in france (2,12,14) → {6,10}. Then union with japan {12,15} → {6,10,12,15}. Always walk the sorted lists with two pointers.

Numerical 3 · Query optimisation order

● SETUP

Recommend a processing order for (tangerine OR trees) AND (marmalade OR skies) AND (kaleidoscope OR eyes), given postings sizes: eyes 213312, kaleidoscope 87009, marmalade 107913, skies 271658, tangerine 46653, trees 316812.

OR group	Estimated size (sum of df)	Order
kaleidoscope OR eyes	$87009 + 213312 = 300321$	1st (smallest)
marmalade OR skies	$107913 + 271658 = 379571$	2nd
tangerine OR trees	$46653 + 316812 = 363465$	—

PROCESS ORDER: (kaleidoscope OR eyes) → (tangerine OR trees) → (marmalade OR skies)

● INSIGHT

Estimate each OR by the SUM of its terms' document frequencies, then AND them in increasing order of that estimate: $300321 < 363465 < 379571$. Smallest candidate set first minimises work.

Practice · Boolean evaluation

● QUESTION

Given precision = B/C and recall = B/A : a system returns 20 documents, 12 of which are relevant, and the collection contains 30 relevant documents in total. Find precision and recall.

● MODEL ANSWER

$B = 12$, $C = 20$, $A = 30$. Precision = $12/20 = 0.60$ (60%). Recall = $12/30 = 0.40$ (40%).

Practice · DNF query

● QUESTION

Terms in order (park, mountain, trails, difficult). Query in DNF: $Q = (1,0,1,0) \text{ OR } (0,1,1,0) \text{ OR } (1,1,1,0)$. Describe what Q asks for.

● MODEL ANSWER

Q matches documents that contain trails AND ((park, not mountain) OR (mountain, not park) OR (both park and mountain)) — i.e. trails present, difficult absent, and at least one of park/mountain present. Each tuple is one allowed term-presence pattern; OR unions them.

03

MODULE 03

Preprocessing & Postings

CS3 · Tokens, stemming, skip lists, phrases

3.1 From documents to terms — the pipeline

Before indexing, raw text is turned into a clean vocabulary of terms through four stages:

- 1 **Tokenisation** — chop text into tokens.
- 2 **Stop-word removal** — drop high-frequency, low-value words.
- 3 **Normalisation** — canonicalise tokens into equivalence classes.
- 4 **Stemming / lemmatisation** — reduce to root / dictionary form.

3.2 Tokenisation ★★

Tokenisation chops a document into **tokens** by removing unwanted characters. “Friends, Romans, Countrymen” → [Friends] [Romans] [Countrymen]. Three terms to keep distinct:

Token An **instance** of a character sequence grouped as a useful semantic unit in a document.

Type The **class** of all tokens with the same character sequence.

Term A (normalised) type that is **included in the IR dictionary**.

● COUNTING EXAMPLE

“She was young the way an actual young person is young.” → **11 tokens, 9 types, 5 terms** (after stop-words & duplicates removed). Watch for this counting question.

Tokenisation issues

- **Apostrophes & possessives:** Cooper's vs Cooper vs Coopers; O'Neill, aren't.
- **Hyphens:** Hewlett-Packard, state-of-the-art, co-education — one token or several?
- **Whitespace:** San Francisco — one token or two?
- **Special sequences:** emails, URLs, IP addresses, tracking numbers — embedded spaces, often useful.
- **Language-specific:** French l'ensemble; German compounds unsegmented; Chinese/Japanese have no spaces; Arabic/Hebrew right-to-left with embedded left-to-right numbers.

3.3 Stop words

Common words of little value in selecting documents (the, a, and, to, be). Build a **stop-list** by sorting terms on **collection frequency** and taking the most frequent. Removal usually helps, but hurts **phrase queries** (“King of Denmark”, “To be or not to be”) and relational queries (“flights to London”). Modern systems use small or no stop-lists because good compression and query optimisation make keeping them cheap.

3.4 Normalisation

Token normalisation canonicalises tokens so matches occur despite surface differences (U.S.A. ↔ USA). The standard approach builds **equivalence classes**, named after one member.

- Delete periods (U.S.A. → USA); delete hyphens (anti-discriminatory → antidiscriminatory).
- Synonyms/homonyms via hand-built classes (car = automobile) or query expansion.

- **Case folding** — usually lowercase everything (with care for mid-sentence capitals / acronyms).
- Accents & diacritics — résumé vs resume.
- Asymmetric expansion: enter *window* → search window, windows; enter *Windows* → Windows only.

3.5 Stemming vs. Lemmatisation ★★

STEMMING

- ▶ Crude heuristic, chops word endings
- ▶ Collapses derivationally related words
- ▶ Language-dependent, fast
- ▶ automate, automatic, automation → automat

LEMMATISATION

- ▶ Full morphological analysis + vocabulary
- ▶ Returns the dictionary form (lemma)
- ▶ Removes inflectional endings only
- ▶ am, are, is → be

Porter's algorithm

The most common English stemmer: **5 phases** of reductions applied sequentially, each with rules selected by conventions (e.g. apply the rule matching the **longest suffix**). Some rules are weight-sensitive, e.g. ($m > 1$) EMENT → ∅ so replacement → replac but cement → cement. Other stemmers: Lovins (single-pass, ~250 rules), Paice/Husk, Snowball.

3.6 Faster merges — skip pointers ★★

Augment a postings list with **skip pointers**: shortcuts that let the merge skip over postings that can't be in the result. During intersection, if the skip target is still $\leq$ the other list's current docID, jump to it instead of stepping one-by-one.

Placemen For a postings list of length P , use $\sqrt{P}$ **evenly-spaced** skips (simple heuristic).
t

Trade-off More skips → shorter spans, more chances to skip but more comparisons. Fewer skips → few comparisons but long spans and few successful skips.

● LIMITATION

Skip pointers are easy on a **static** index but awkward when P keeps changing due to updates. They only help **AND** (intersection), not OR.

3.7 Phrase queries ★★

"Stanford University" as a phrase must *not* match "I went to university at Stanford". Plain $\langle \text{term} : \text{docs} \rangle$ postings are insufficient. Two solutions:

Biword index

- Index every consecutive pair of terms as one dictionary term: "Friends Romans Countrymen" → {friends romans, romans countrymen}.
- Two-word phrase lookup is immediate; longer phrases broken into a Boolean AND of biwords (stanford university AND university palo AND palo alto).
- **Extended biwords**: POS-tag, keep nouns (N) and articles/prepositions (X), index strings of form NX^*N .
- **Problems**: false positives (can't verify the phrase without positions), and big dictionary blowup.

Positional index ★★

Store, for each term and document, the **positions** where it occurs:

$\langle \text{term, doc-freq; doc1: tf; pos1, pos2 } \dots; \text{ doc2: tf; pos1, pos2 } \dots \rangle$

Phrase queries merge recursively at the document level, then check that positions are **compatible** (consecutive offsets). The same index also answers **proximity** queries. Cost: much larger index than a plain inverted index.

04

MODULE 04

Dictionaries & Tolerant Retrieval

CS4/CS5 · Structures, wildcards, spelling, Soundex

4.1 Dictionary data structures

The dictionary stores the vocabulary, document frequencies and postings pointers. The choice of structure depends on the number of keys, whether they are static or dynamic, and access frequencies.

Structure	Lookup	Prefix search?	Notes
Hashtable	$O(1)$	No	Fastest; no minor-variant or prefix search; rehash as vocab grows
Binary search tree	$O(\log M)$	Yes	Needs balance for $O(\log M)$; rebalancing is expensive
B-tree	$O(\log M)$	Yes	Internal nodes have children in $[a,b]$; mitigates rebalancing

● THE PREFIX PROBLEM

Hashtables can't find all terms starting with "automat". Trees can — this is exactly why wildcard queries need a tree-based (or specialised) index.

4.2 Tolerant retrieval — overview

Techniques robust to typos and alternative spellings, in three families:

Wildcard queries handle * in the query term (Sydney vs Sidney → S*dney).

Spelling correction find the nearest correct word(s) to a misspelling.

Phonetic correction match terms that *sound* alike (Soundex).

4.3 Wildcard queries ★★

- **Trailing** (mon*): on a B-tree, retrieve the range $\text{mon} \leq w < \text{moo}$.
- **Leading** (*mon): keep a **reverse B-tree** (terms written backwards), retrieve $\text{nom} \leq w < \text{non}$.
- **Middle** (co*tion): look up co* AND *tion and intersect — expensive. Better: transform so the * is always at the end → the **permuterm index**.

Permuterm index ★★

Add an end-marker \$ and index every rotation of the term+\$. For **hello**: hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello. To answer a wildcard, rotate the query so the * is at the end, then do a prefix lookup.

X* search \$X*

***X** search X\$*

X search X*

X*Y search Y\$X*

● WORKED ROTATION

Query **m*n**: append \$ → m*n\$, rotate so * is last → **n\$m***. Prefix-search n\$m* in the permuterm index to get matching terms, then fetch their postings.

k-gram index ★★

A **k-gram** is a sequence of k characters. For **castle** the 3-grams (with \$ boundary) are \$ca, cas, ast, stl, tle, le\$. The k-gram index maps each k-gram → all terms containing it.

- Run mon* as **\$m AND mo AND on** over the bigram index.
- Returns a **superset** — e.g. “moon” is a false positive — so **post-filter** survivors against the original wildcard, then look them up in the inverted index.
- Fast and space-efficient.

4.4 Spelling correction ★★★

Among alternative correct spellings of a misspelling, pick the **nearest**; break ties toward the **more common** word. Two forms:

ISOLATED WORD

- ▶ Check each word on its own
- ▶ Misses real-word typos
- ▶ e.g. “from” typed as “form” slips through

CONTEXT-SENSITIVE

- ▶ Look at surrounding words
- ▶ Catches real-word errors
- ▶ “I flew form Heathrow” → from

Isolated correction needs a **lexicon** (Webster's, an industry list, or the indexed corpus) and a closeness measure: **edit distance**, **weighted edit distance**, or **k-gram overlap**.

Edit (Levenshtein) distance ★★★

The minimum number of character operations — **insert**, **delete**, **replace** (optionally **transpose**) — to turn one string into another. Computed by **dynamic programming**: a matrix m where m(i, j) is the edit distance between the first i chars of s1 and first j of s2.

$$m(i, j) = \min[m(i-1, j)+1, m(i, j-1)+1, m(i-1, j-1) + (0 \text{ if } s1_i=s2_j \text{ else } 1)]$$

dof → dog 1 (one replace)

cat → act 2 (or 1 with transpose)

cat → dog 3 (three replaces)

Weighted edit distance

Like edit distance, but the cost of an operation depends on the characters — capturing OCR / keyboard errors (m mis-typed as n is cheaper than as q). Needs a weight matrix; modify the DP to use weights. Can be framed as a probability model.

k-gram overlap & Jaccard ★★

Computing edit distance to every dictionary term is too slow. Instead, use the k-gram index to fetch candidate terms sharing enough k-grams, then threshold by overlap.

$$\text{Jaccard}(X, Y) = |X \cap Y| / |X \cup Y|$$

- Always between 0 and 1; = 1 when identical, = 0 when disjoint; X and Y need not be the same size.
- Threshold to decide a match (e.g. J.C. > 0.8).
- Example: November trigrams {nov, ove, vem, emb, mbe, ber} vs December {dec, ece, cem, emb, mbe, ber} share 3 → overlap measure built from these.

Context-sensitive correction

For “flew form munich”: **hit-based** correction enumerates close terms for each word, tries phrases fixing one word at a time (flea form munich / flew from munich / flew form munch), and picks the phrase with the **most hits** — “flew from munich”. Alternative: break into biwords, correct the one needing a single fix, rank the matches.

4.5 Phonetic correction — Soundex ★★

For misspellings that **sound like** the target. Generate a **phonetic hash** so similar-sounding terms collide. Soundex (US census, 1918) is the classic, mainly for **names**.

Soundex algorithm — 4 chars

- 1 **Retain the first letter** of the word.
- 2 Map A, E, I, O, U, H, W, Y → 0. Map consonants to digits: B F P V → 1 · C G J K Q S X Z → 2 · D T → 3 · L → 4 · M N → 5 · R → 6.
- 3 **Collapse** one of each pair of consecutive identical digits.
- 4 **Remove all zeros**, then pad/truncate to the form <letter><digit><digit><digit>.

● SOUNDEX CAVEATS

Provided by most databases (Oracle, SQL Server) but **not much use for IR**; okay for high-recall tasks (Interpol) though biased toward certain nationalities of names.

N2

MODULE N2

Worked Numericals · Tolerant

Skip pointers, edit distance, Soundex, permuterm

Numerical 4 · Skip-pointer comparisons

● SETUP

List A (16 elements): 4 ... 47 ... 120, with skip pointers at $\sqrt{16} = 4$ spacing, i.e. at positions holding 4, 14, 22, 120. Search for element 47. Count comparisons with vs without skips.

Method	Comparisons made	Count
Without skips	step linearly 4 → ... → 47 (all elements up to 47)	11
With skips ($\sqrt{P}=4$)	4·47, skip→14·47, skip→22·47, skip→120·47 (overshoot), 32·47, 47·47	6

SKIPS WIN: 6 comparisons vs 11

● INSIGHT

Skip length $\sqrt{P}$ places pointers at 4, 14, 22, 120. We skip while the skip target ≤ 47 ; once 120 overshoots, we step linearly from 22 (→32→47). Skips cut 11 comparisons to 6.

Numerical 5 · Edit distance (Levenshtein DP)

● SETUP

Compute the edit distance between kitten and sitting by filling the DP matrix (insert/delete/replace, cost 1 each). Read the answer from the bottom-right cell.

	ε	s	i	t	t	i	n	g
ε	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

EDIT DISTANCE = 3 (k→s, e→i, insert g)

● INSIGHT

Each cell = $\min(\text{left}+1, \text{top}+1, \text{diagonal} + [0 \text{ if chars match else } 1])$. The diagonal of matching letters (i, t, t) keeps costs low; the final 3 = two substitutions plus one insertion.

Numerical 6 · Soundex code

● SETUP

Compute the Soundex code for Robert and Rupert. Do they collide?

Step	Robert	Rupert
Keep first letter	R	R
Map rest to digits	R o(0) b(1) e(0) r(6) t(3)	R u(0) p(1) e(0) r(6) t(3)
Drop zeros	R 1 6 3	R 1 6 3
Pad / truncate to 4	R163	R163
Both → R163 (they collide — phonetically equivalent)		

● INSIGHT

Soundex maps similar-sounding names to the same 4-char code. Likewise Herman and Hermann both give H655 (the double-n collapses). This is the whole point of a phonetic hash.

Practice · Permuterm rotation

● QUESTION

Using a permuterm index, what rotated string do you look up to answer the wildcard query `fi*er`, and what do you do with the results?

● MODEL ANSWER

Append `$` → `fi*er$`, then rotate so `*` is last → `er$fi*`. Prefix-search `er$fi*` in the permuterm index to get candidate terms (e.g. `filer`, `fixer`, `finisher`...), then look those terms up in the inverted index to retrieve documents.

Practice · k-gram Jaccard

● QUESTION

Query trigram set has 6 trigrams; a candidate term has 6 trigrams; they share 4. Compute the Jaccard coefficient. Would a 0.8 threshold accept it?

● MODEL ANSWER

$|X \cap Y| = 4$, $|X \cup Y| = 6 + 6 - 4 = 8$. Jaccard = $4/8 = 0.5$. Since $0.5 < 0.8$, it would NOT be accepted as a match at that threshold.

Practice · Edit distance

● **QUESTION**

What is the (unweighted) edit distance from cat to act, and how does it change if transposition is allowed as a single operation?

● **MODEL ANSWER**

Without transposition: 2 (e.g. replace $c \rightarrow a$ and $a \rightarrow c$, or delete+insert). With transposition allowed: 1 (swap the first two letters).

05

Index Construction

CS6 · BSBI, SPIMI, MapReduce, dynamic indexing

5.1 Why hardware matters

Index design follows hardware reality. Memory access is far faster than disk; **disk I/O is block-based** (8 KB–256 KB), and no data transfers while the head seeks. So transferring one **large** chunk beats many small chunks. Fault tolerance is expensive — IR uses many commodity machines rather than one reliable one.

Symbol	Statistic	Value
s	average disk seek time	5 ms = 5×10^{-3} s
b	transfer time per byte	$0.02 \mu\text{s} = 2 \times 10^{-8}$ s
p	low-level operation (compare/swap)	$0.01 \mu\text{s} = 10^{-8}$ s
—	main memory / disk	several GB / 1 TB+

● REUTERS RCV1 (THE RUNNING EXAMPLE)

$N = 800,000 \text{ docs} \cdot \sim 200 \text{ tokens/doc} \cdot M \approx 400,000 \text{ terms} \cdot 100,000,000 \text{ non-positional postings}$. Too big to sort in memory → need external algorithms.

5.2 The problem

In-memory index construction **does not scale** — you can't load the whole collection, sort, and write back. Sorting 100M postings records directly on disk is also too slow (too many seeks). We need an **external sorting** algorithm that minimises disk seeks.

5.3 BSBI — Blocked Sort-Based Indexing ★★

Postings are 12-byte (termID, docID, freq) records generated as we parse. Idea:

- 1 Split the stream into **blocks** (e.g. 10M records each) that fit in memory.
- 2 For each block: accumulate postings, **sort in memory** (quicksort, $\sim 2N \ln N$), write the sorted run to disk.
- 3 **Merge** all the sorted runs into one final index — ideally a **multi-way merge** reading from all runs at once.

● BSBI'S LIMITATION

It needs the whole **dictionary** (term ↔ termID mapping) in memory to generate termIDs. Working with term-strings instead avoids the mapping but blows up the intermediate file sizes.

5.4 SPIMI — Single-Pass In-Memory Indexing ★★

SPIMI removes both BSBI pain points with two ideas:

- **Key idea 1:** generate a **separate dictionary per block** — no global term↔termID mapping needed.
- **Key idea 2: don't sort** — accumulate postings directly into postings lists as terms occur.

Each block becomes a complete inverted index; the per-block indexes are then merged (like BSBI). Compression of terms and postings makes SPIMI even more efficient.

	BSBI	SPIMI
Sorting	Yes — sort each block	No — append to postings lists
Dictionary	Global term $\leftrightarrow$ termID needed	Separate dictionary per block
Per-block output	Sorted runs of records	Complete inverted index

5.5 Distributed indexing & MapReduce ★★

For web scale, use a cluster of fault-prone commodity machines coordinated by a **master** (considered safe), which assigns parallel tasks to idle machines. Two task types:

Parsers read a document split, emit (term, doc) pairs, write them into j partitions by term-range (e.g. a-f, g-p, q-z).

Inverters collect all (term, doc) pairs for one term-partition, sort, and write the postings lists.

This is exactly an instance of **MapReduce** (Dean & Ghemawat, 2004):

```
map: collection  $\rightarrow$  list(termID, docID) reduce: (termID, list(docID))  $\rightarrow$  postings
list
```

● PARTITIONING

Term-partitioned: one machine handles a range of terms. **Document-partitioned**: one machine handles a range of docs. Most search engines use document-partitioned indexes — better load balancing.

5.6 Dynamic indexing ★★

Real collections change — docs are added, deleted, modified. The dictionary and postings must update.

Simplest approach: main + auxiliary

- Keep a big **main index** plus a small in-memory **auxiliary index** for new docs; search both and merge.
- Deletions handled by an **invalidation bit-vector** filtering results.
- Periodically re-index everything into one main index.
- **Problem**: frequent merges are costly; one big index file makes each merge touch everything $\rightarrow O(T^2)$.

Logarithmic merge ★★

Maintain a series of indexes, each twice the size of the previous (powers of 2). Keep the smallest Z_0 in memory; when it overflows, merge up the chain ($Z_0 \rightarrow I_0$, or merge to form $Z_1 \rightarrow I_1, \dots$).

Scheme	Construction cost	Query cost
Main + auxiliary	$O(T^2)$ — each posting touched every merge	$O(1)$ — merge 2 indexes
Logarithmic merge	$O(T \log T)$ — each posting merged $O(\log T)$ times	$O(\log T)$ — merge $\log T$ indexes

● **TRADE-OFF**

Logarithmic merge is far cheaper to **build** but query processing must merge $O(\log T)$ indexes. Collection-wide statistics (e.g. for spell-correction “most hits”) are also harder to maintain across multiple indexes.

06

MODULE 06

Index Compression

CS7 · Heaps, Zipf, dictionary & postings compression

6.1 Why compress?

- Use less disk (saves money) and keep **more in memory** (faster).
- Speed up disk→memory transfer: [read compressed | decompress] beats [read uncompressed], because modern **decompression is fast**.
- Lets large search engines hold a big slice of the postings in RAM.

Lossless all information preserved — the default in IR.

Lossy discards information; preprocessing steps (case-folding, stop-words, stemming, number removal) are effectively lossy compression.

6.2 Vocabulary growth — Heaps' law ★★

How big does the vocabulary get? There is **no** fixed upper bound — it keeps growing with the collection. Heaps' law estimates vocabulary size M from token count T :

$$M = k \cdot T^b \text{ (typically } 30 \leq k \leq 100, b \approx 0.5)$$

It is the simplest relationship in log-log space, an **empirical law**. For RCV1 the best fit is $\log_{10} M = 0.49 \cdot \log_{10} T + 1.64$, so $k \approx 44$ and $b = 0.49$ — predicting 38,323 terms for the first ~1M tokens vs 38,365 actual. Implication: the dictionary **keeps growing** and is large for big collections.

6.3 Term distribution — Zipf's law ★★

A few terms are very frequent; very many are rare. Zipf's law: the i -th most frequent term has collection frequency proportional to $1/i$.

$$cf_i \propto 1/i \text{ i.e. } cf_i = K/i \rightarrow \log cf_i = \log K - \log i$$

So the most frequent term (the) occurs cf_1 times, the second (of) $cf_1/2$, the third (and) $cf_1/3$, ... — a linear relationship between $\log cf_i$ and $\log i$ (another **power law**).

● HEAPS VS ZIPF — DON'T MIX THEM UP

Heaps = how the *vocabulary size* grows with collection size. **Zipf** = how *term frequencies* are distributed within the collection.

6.4 Dictionary compression ★★

A fixed-width array (20 bytes/term + 4 + 4) wastes space: most words are short but we allot 20 bytes, and still can't fit very long words. English words average ~8 characters.

Technique	Idea	RCV1 size
Fixed width	20-byte term field per entry	11.2 MB
Dictionary-as-a-string	Concatenate all terms; store a pointer per term (3 B) to find boundaries	7.6 MB

+ Blocking (k)	Store a pointer only every k-th term; add 1 length byte per term	7.1 MB (k=4)
+ Front coding	Sorted terms share long prefixes; store prefix once + suffix differences	5.9 MB

Blocking trades pointer bytes for length bytes: for block size k you save $(k-1) \times 3$ pointer bytes but spend k length bytes. **Front coding** example: 8automat*a1e2ic3ion encodes automate / automatic / automation as a shared prefix “automat” plus suffix deltas.

● BLOCKING & SEARCH

Blocking slightly **slows lookup** (binary search to the block, then linear scan inside it) — e.g. avg ~ 3 comparisons with k=4 vs ~ 2.6 without — but saves space.

6.5 Postings compression ★★★

The postings file is $\geq 10\times$ the dictionary. A raw docID for RCV1 needs 32 bits (or $\log_2 800,000 \approx 20$ bits). Goal: **far fewer than 20 bits** per posting. Two opposing forces: a rare term (arachnocentric) wants ~ 20 bits; a ubiquitous term (the) wants ~ 1 bit (a bitmap).

Gap encoding

Store postings in increasing docID order, then store the **gaps** between consecutive docIDs instead of absolute IDs. Gaps for frequent terms are small $\rightarrow$ few bits.

33, 47, 154, 159, 202 $\rightarrow$ store gaps 33, 14, 107, 5, 43

If the average gap for a term is G, we want $\sim \log_2 G$ bits per entry $\rightarrow$ a **variable-length** code.

Variable Byte (VB) codes ★★★

- 1 Use 7 bits of each byte for the number and 1 bit as a **continuation bit** c.
- 2 If $G \leq 127$, encode it in 7 bits and set $c = 1$ (this is the last byte).
- 3 Otherwise encode the low-order 7 bits, recurse on the higher bits in more bytes.
- 4 Set $c = 1$ on the **last** byte, $c = 0$ on all the others. Result is uniquely prefix-decodable.

● RCV1 POSTINGS SIZES

uncompressed 32-bit = 400 MB · uncompressed 20-bit = 250 MB · **Variable Byte = 116 MB** · y-encoded = 101 MB. Compression more than halves the postings.

N3

MODULE N3

Worked Numericals · Compression

Blocking, Variable Byte, Heaps

Numerical 7 · Dictionary blocking savings

● SETUP

RCV1 dictionary-as-string baseline = 7.6 MB, 400,000 terms, 3-byte term pointers. Compute the space with blocking for $k = 8$ and $k = 16$.

k	Saved/block (pointers – lengths)	Total reduction	Size
4	$(4-1) \cdot 3 - 4 = 5$ bytes / 4 terms	$400000 \cdot 5/4 = 0.5$ MB	7.1 MB
8	$(8-1) \cdot 3 - 8 = 13$ bytes / 8 terms	$400000 \cdot 13/8 = 0.65$ MB	6.95 MB
16	$(16-1) \cdot 3 - 16 = 29$ bytes / 16 terms	$400000 \cdot 29/16 = 0.725$ MB	6.875 MB

Larger $k \rightarrow$ more saving, but slower in-block search

● INSIGHT

Per block you save $(k-1) \cdot 3$ pointer bytes but add k length bytes; net = $(k-1) \cdot 3 - k$ per block, times $400000/k$ blocks. Diminishing returns set in as k grows.

Numerical 8 · Variable Byte encode

● SETUP

Encode the gap sequence for docIDs 824, 829, 215406. First take gaps, then VB-encode (7 data bits/byte, high bit = continuation, 1 on last byte).

docID	gap	VB code (bytes)
824	824	00000110 10111000
829	5	10000101
215406	214577	00001101 00001100 10110001

Concatenated stream: 00000110 10111000 10000101 00001101 00001100 10110001

● INSIGHT

$824 = 6 \cdot 128 + 88 \rightarrow$ two bytes; the last byte's high bit is 1 (10111000), the first is 0 (00000110). $5 \leq 127 \rightarrow$ single byte 10000101. The stream is uniquely decodable by reading until a high bit of 1.

Numerical 9 · Variable Byte decode

● SETUP

Decode the VB stream 10001001 00000011 10000010 11111110 into its gap (posting) list.

Byte(s)	Continuation bits	Value
10001001	high bit 1 → ends here	$0001001_2 = 9$
00000011 10000010	0 then 1 → two-byte number	$0000011 \cdot 128 + 0000010 = 386$
11111110	high bit 1 → single byte	$1111110_2 = 126$
Gap list = 9, 386, 126 (docIDs = 9, 395, 521)		

● INSIGHT

Group bytes until you hit one with high bit = 1; that closes a number. Strip the continuation bits and concatenate the 7-bit payloads. The values are gaps; cumulative-sum them for docIDs.

Practice · Heaps' law

● QUESTION

With Heaps' law $M = k \cdot T^b$ and the RCV1 fit $k = 44$, $b = 0.49$, roughly how many terms are predicted for $T = 1,000,000$ tokens?

● MODEL ANSWER

$M = 44 \cdot (10^6)^{0.49} = 44 \cdot 10^{(0.49 \cdot 6)} = 44 \cdot 10^{2.94} \approx 44 \cdot 871 \approx 38,300$ terms — close to the ~38,365 actually observed.

Practice · Largest 2-byte VB value

● QUESTION

What is the largest gap that can be encoded in exactly 2 bytes with Variable Byte encoding?

● MODEL ANSWER

Two bytes give $7 + 7 = 14$ usable data bits, so the maximum value is $2^{14} - 1 = 16383$.

07

Vector Space Model

CS8 · tf-idf weighting and cosine scoring

7.1 Why move beyond Boolean?

Boolean queries match or don't — good for experts, bad for the majority. Most users can't write Boolean expressions and don't want to wade through thousands of unranked results. **Ranked retrieval** returns an **ordering** of documents; show only the top k (≈ 10). Each document gets a **score** (say in $[0,1]$) measuring how well it matches the query.

7.2 Jaccard for scoring — and why it fails

$$\text{jaccard}(A, B) = |A \cap B| / |A \cup B|$$

A simple overlap measure between query set and document set (0–1, size-independent). But it has three fatal flaws for ranking:

- Ignores **term frequency** — how often a term occurs in the document.
- Ignores that **rare terms are more informative** than common ones.
- Doesn't normalise **document length** well.

7.3 Term frequency — bag of words ★★

Represent each document as a **count vector** over the vocabulary. Word **order is lost** — “John is quicker than Mary” and “Mary is quicker than John” get identical vectors. This is the **bag of words** model.

Raw term frequency $tf_{t,d}$ isn't used directly: a doc with 10 occurrences is more relevant than one with 1, but **not 10x more**. So we dampen it with a log:

$$w_{t,d} = 1 + \log_{10}(tf_{t,d}) \text{ if } tf > 0, \text{ else } 0$$

So $tf \ 0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$. A document's score for a query sums $w_{t,d}$ over the terms common to query and document.

7.4 Inverse document frequency ★★★

df_t = number of documents containing t — an *inverse* measure of informativeness. Rare terms (arachnocentric) deserve high weight; frequent terms (good, increase) low but positive weight. Define:

$$\text{idf}_t = \log_{10}(N / df_t) \text{ (} N = \text{number of documents)}$$

● IDF AND ONE-TERM QUERIES

idf has **no effect** on ranking single-term queries (it scales all candidate docs equally). It only matters for queries with ≥ 2 terms.

7.5 tf-idf weighting ★★★

The best-known weighting scheme in IR — the product of the tf weight and the idf weight:

$$w_{t,d} = (1 + \log tf_{t,d}) \times \log(N / df_t)$$

- Increases with the number of occurrences **within** a document (tf).
- Increases with the **rarity** of the term across the collection (idf).
- (The “-” in tf-idf is a hyphen, not a minus; also written tf·idf or tf × idf.)

7.6 Documents & queries as vectors ★★

Each document becomes a real-valued vector of tf-idf weights in $\mathbb{R}^{|V|}$ — terms are axes, documents are points. Vectors are very high-dimensional and very **sparse**. Represent the query the same way, then rank documents by **proximity** to the query vector.

Why angle, not distance

Euclidean distance is misleading: append a document to itself (d') and it has the same meaning, but its Euclidean distance from the original is large. The **angle** between them is 0 (maximal similarity). So rank by the angle — equivalently, by its **cosine** (cosine decreases monotonically over 0° – 180° , so smallest angle = largest cosine).

Cosine similarity ★★★

$$\cos(q, d) = (q \cdot d) / (|q| \cdot |d|) = \sum_i q_i d_i / (\sqrt{\sum_i q_i^2} \cdot \sqrt{\sum_i d_i^2})$$

The denominator is **length (L2) normalisation** — dividing by the L2 norm makes each vector a unit vector, so long and short documents become comparable (d and d' become identical). For **length-normalised** vectors, cosine is just the **dot product**.

● SMART NOTATION DDD.QQQ

Each triple is (term-frequency · document-frequency · normalisation). **Inc.Itn** = document: log tf, no idf, cosine; query: log tf, tidf, no normalisation.

7.7 Ranking summary & limitations

- 1 Represent the query as a weighted tf-idf vector.
- 2 Represent each document as a weighted tf-idf vector.
- 3 Compute the cosine similarity between query and each document.
- 4 Rank documents by descending score; return the top K (≈ 10).

● VSM LIMITATIONS

Missing semantics (word sense) and syntax (order, proximity).

Assumes term independence; ignores synonymy.

Lacks Boolean control (can't force a term to appear).

May prefer a doc rich in A-but-no-B over one with both A and B less frequently.

N4

MODULE N4

Worked Numericals · VSM

idf, cosine, the Inc.ltn table

Numerical 10 · idf values

● SETUP

With $N = 1,000,000$ documents and base-10 logs, compute $\text{idf} = \log_{10}(N/\text{df})$ for each term.

Term	df _t	idf _t = $\log_{10}(10^6/\text{df})$
calpurnia	1	$\log 10^6 = 6$
animal	100	$\log 10^4 = 4$
sunday	1,000	$\log 10^3 = 3$
fly	10,000	$\log 10^2 = 2$
under	100,000	$\log 10^1 = 1$
the	1,000,000	$\log 10^0 = 0$

● INSIGHT

A term in every document (the, $\text{df} = N$) gets $\text{idf} = 0$ — no discriminating power. A term in one document gets the maximum idf. Each $\times 10$ in df drops idf by 1 (base-10 log).

Numerical 11 · Cosine similarity between documents

● SETUP

Term counts: SaS (affection 115, jealous 10, gossip 2, wuthering 0), PaP (58, 7, 0, 0), WH (20, 11, 6, 38). Use log-frequency weights then length-normalise (no idf). Find $\cos(\text{SaS}, \text{PaP})$.

Term	SaS 1+log tf → n'lized	PaP 1+log tf → n'lized	product
affection	$3.06 \rightarrow 0.789$	$2.76 \rightarrow 0.832$	0.657
jealous	$2.00 \rightarrow 0.515$	$1.85 \rightarrow 0.555$	0.286
gossip	$1.30 \rightarrow 0.335$	$0 \rightarrow 0$	0
wuthering	$0 \rightarrow 0$	$0 \rightarrow 0$	0
$\cos(\text{SaS}, \text{PaP}) = 0.789 \cdot 0.832 + 0.515 \cdot 0.555 \approx 0.94$			

● INSIGHT

Normalise each column by its L2 norm (e.g. SaS: $\sqrt{(3.06^2+2^2+1.30^2)} = 3.87$), then the cosine is the dot product of the unit vectors. Similarly $\cos(\text{SaS}, \text{WH}) \approx 0.79$ and $\cos(\text{PaP}, \text{WH}) \approx 0.69$ — SaS and PaP are the most alike.

Numerical 12 · Full Inc.ltn scoring ★★★

● SETUP

Query “best car insurance”, Document “car insurance auto insurance”. $N = 10^6$. Doc = Inc (log tf, no idf, cosine); Query = ltn (log tf, idf, no normalisation). df: auto 5000, best 50000, car 10000, insurance 1000.

Term	q tf-wt	idf	q wt	d tf-wt	d n'lize	product
auto	0	2.3	0	1	0.52	0
best	1	1.3	1.3	0	0	0
car	1	2.0	2.0	1	0.52	1.04
insurance	1	3.0	3.0	1.3	0.68	2.04

FINAL SIMILARITY SCORE = $0 + 0 + 1.04 + 2.04 = 3.08$

● INSIGHT

Query weight = $(1+\log \text{tf}) \cdot \text{idf}$, left UN-normalised (ltn). Document weight = $(1+\log \text{tf})$, then cosine-normalised by $\sqrt{(1^2+0^2+1^2+1.3^2)} = \sqrt{3.69} \approx 1.92$ (Inc). Multiply matched terms and sum. auto and best contribute 0 because each is absent from one side.

Practice · idf effect

● QUESTION

For the one-term query “iPhone”, does idf change the document ranking? Why?

● MODEL ANSWER

No. idf multiplies every candidate document's score by the same constant (the idf of iPhone), so the relative order is unchanged. idf only affects ranking for queries of two or more terms.

Practice · log tf weight

● QUESTION

A term occurs 1000 times in a document. What is its log-frequency weight $1 + \log_{10}(\text{tf})$?

● MODEL ANSWER

$1 + \log_{10}(1000) = 1 + 3 = 4$. (Compare: $\text{tf} = 10$ gives weight 2, $\text{tf} = 2$ gives 1.3 — relevance grows far slower than raw count.)

**MODULE R**

Cheat Sheet

Formulas, algorithms, traps & checklist

R1 Key formulas

Formula	Used for
Precision = B/C , Recall = B/A	Boolean / set evaluation (B =rel.retrieved, C =retrieved, A =relevant)
postings merge = $O(m + n)$	intersecting two sorted postings lists
skip spacing = $\sqrt{P}$	where to place skip pointers on a length- P list
$m(i,j)=\min(\text{left}+1, \text{up}+1, \text{diag}+\delta)$	Levenshtein edit-distance DP ($\delta=0$ if equal else 1)
Jaccard = $ X \cap Y / X \cup Y $	k-gram overlap; set similarity (0-1)
$M = k \cdot T^b$ ($k \approx 44$, $b \approx 0.5$)	Heaps' law — vocabulary size vs tokens
$cf_i \propto 1/i$	Zipf's law — term-frequency distribution
blocking net = $(k-1) \cdot 3 - k$ per block	dictionary blocking space saving
$w_{t,d} = 1 + \log_{10} tf_{t,d}$	log-frequency term weight
$idf_t = \log_{10}(N / df_t)$	inverse document frequency
$tf-idf = (1 + \log tf) \cdot \log(N/df)$	the standard term weight
$\cos(q,d) = q \cdot d / (q d)$	vector space similarity (= dot product if normalised)

R2 Algorithms at a glance

Inverted index	tokenise → preprocess → (term,docID) pairs → sort → merge → dictionary + postings.
Boolean optimise	AND: smallest df first. OR-groups: process in increasing order of summed df.
Permuterm	add \$, index all rotations; rotate the query so * is at the end, prefix-search.
k-gram wildcard	Boolean-AND the query's k-grams; post-filter false positives.
Edit distance	fill DP grid, answer is bottom-right cell.
Soundex	keep 1st letter; map to digits; collapse repeats; drop 0s; pad to 4 chars.
BSBI	sort each in-memory block, write run, multi-way merge runs.
SPIMI	per-block dictionary, no sort (append to postings), then merge blocks.

MapReduce	map → (termID, docID); reduce → postings lists. Parsers + inverters.
Log merge	indexes in powers of 2; build $O(T \log T)$, query $O(\log T)$.
Variable Byte	7 data bits/byte; high bit = 1 on last byte; encode gaps not docIDs.
VSM ranking	tf-idf vectors → cosine → sort desc → top K.

R3 Common traps

● WATCH OUT

token vs type vs term — count them separately; terms = after stop-words & dedup.

idf needs ≥ 2 query terms to affect ranking; $df=N \Rightarrow idf=0$.

Heaps = vocabulary growth; **Zipf** = frequency distribution. Don't swap them.

VB encodes **gaps**, not raw docIDs; continuation bit = 1 marks the **last** byte.

Permuterm: the * must be moved to the **end** after adding \$.

Cosine ranks by **angle**; Euclidean distance is misleading for different-length docs.

Skip pointers help **AND** only, and assume a fairly static index.

R4 Exam-day checklist

- Closed book, 2 hours, covers sessions 1–8 (CS1–CS8) — manage ~15 min per major question.
- For any algorithm question: **draw the trace** (DP grid / postings merge / tf-idf table).
- State formulas before plugging numbers; carry units and bases ($\log_{10}$ unless told otherwise).
- For compare questions, use a 2-column table (advantages/limitations, or X vs Y).
- Sanity-check: $idf \geq 0$, Jaccard $\in [0,1]$, cosine $\in [0,1]$ for non-negative weights, gaps > 0 .